

Exercícios — Aula 03 — Pilhas e filas com ArrayDeque

Técnicas de Programação - 2026/02

Última atualização: September 2, 2026

Instruções

- Use **Pilha** para comportamento LIFO: último a entrar, primeiro a sair.
- Use **Fila** para comportamento FIFO: primeiro a entrar, primeiro a sair.
- Em todas as simulações, mostre o estado da estrutura após cada operação.

Exercício 1 — A mesma sequência em pilha e em fila

Simule os dois blocos de pseudocódigo. Compare as saídas.

Algorithm 1: TestePilha

Result: Executa TESTE-PILHA.

Input : none

Output: textos impressos

```
P ← PILHA-VAZIA()
EMPILHAR(P, "Ana")
EMPILHAR(P, "Bruno")
EMPILHAR(P, "Carla")
IMPRIMIR(TOPO(P))
IMPRIMIR(DESEMPILHAR(P))
EMPILHAR(P, "Diego")
IMPRIMIR(DESEMPILHAR(P))
IMPRIMIR(DESEMPILHAR(P))
```

Algorithm 2: TesteFila

Result: Executa TESTE-FILA.

Input : none

Output: textos impressos

```
F ← FILA-VAZIA()
ENFILEIRAR(F, "Ana")
ENFILEIRAR(F, "Bruno")
ENFILEIRAR(F, "Carla")
IMPRIMIR(FRENTE(F))
IMPRIMIR(DESENFILEIRAR(F))
ENFILEIRAR(F, "Diego")
IMPRIMIR(DESENFILEIRAR(F))
IMPRIMIR(DESENFILEIRAR(F))
```

passo	estado da pilha	saída da pilha	estado da fila	saída da fila
1				
2				
3				
4				
5				
6				
7				
8				

Por que as saídas finais são diferentes?

Exercício 2 — Parênteses balanceados

Complete a simulação do algoritmo para `texto = "(()())"` e para `texto = "()()("`.

Algorithm 3: ParentesesBalanceados

Result: Executa PARENTESES-BALANCEADOS.

Input : string

Output: boolean

```
P ← PILHA-VAZIA()
for i ← 0 to TAMANHO(texto) − 1 do
    c ← texto[i]
    if c = "(" then
        EMPILHAR(P, c)
    end
    if c = ")" then
        if VAZIA(P) then
            return false
        end
        DESEMPILHAR(P)
    end
end
return VAZIA(P)
```

i	c	ação	pilha após a ação
0			
1			
2			
3			
4			
5			

Explique o que significa a pilha estar vazia no final.

Exercício 3 — Fila de atendimento

Uma central usa uma fila para registrar chegada e atendimento.

Simule as operações:

1. chega Ana;
2. chega Bruno;
3. atende;
4. chega Carla;
5. chega Diego;
6. atende;
7. consulta próximo;
8. atende.

passo	operação	fila após operação	saída
1			
2			
3			
4			
5			
6			
7			
8			

Qual pessoa fica aguardando ao final?

Exercício 4 — Estrutura inadequada

O algoritmo abaixo usa uma Lista para simular atendimento por ordem de chegada.

Algorithm 4: AtendimentoComLista

Result: Executa ATENDIMENTO-COM-LISTA.

Input : list pessoas

Output: pessoas atendidas

```
while TAMANHO(pessoas) > 0 do  
    pessoa ← OBTER(pessoas, 0)  
    REMOVER(pessoas, 0)  
    IMPRIMIR(pessoa)  
end
```

1. O algoritmo atende na ordem correta?
2. Qual operação tende a ser cara em uma lista baseada em array?
3. Reescreva o algoritmo usando TAD Fila.

Exercício 5 — Pilha ou fila?

Escolha a estrutura mais adequada para cada cenário.

Cenário	Pilha ou fila?	Por quê?
Desfazer a última ação de um editor		
Chamar senhas de atendimento em ordem		
Verificar delimitadores em uma expressão		
Processar tarefas na ordem em que chegaram		
Resolver o próximo item mais recentemente aberto		

Depois, crie um cenário em que usar a estrutura errada muda a resposta.

Exercício 6 — Delimitadores múltiplos

Escreva pseudocódigo para validar textos com os delimitadores (), [] e {}.

Teste com:

- "({})" deve retornar VERDADEIRO;
- "([])" deve retornar FALSO;
- "()" deve retornar FALSO;
- "" deve retornar VERDADEIRO.